EXPLORING SYSTEM ADAPTATIONS FOR MINIMUM LATENCY REAL-TIME PIANO TRANSCRIPTION

Patricia Hu[1] **Silvan David Peter**[1] **Jan Schlüter**[1] **Gerhard Widmer**[1,2]
[1] Institute of Computational Perception, Johannes Kepler University Linz, Austria
[2] LIT AI Lab, Linz Institute of Technology, Austria
patricia.hu@jku.at

ABSTRACT

Advances in neural network design and the availability of large-scale labeled datasets have driven major improvements in piano transcription. Existing approaches target either offline applications, with no restrictions on computational demands, or online transcription, with delays of 128–320 ms. However, most real-time musical applications require latencies below 30 ms. In this work, we investigate whether and how the current state-of-the-art online transcription model can be adapted for real-time piano transcription. Specifically, we eliminate all non-causal processing, and reduce computational load through shared computations across core model components and variations in model size. Additionally, we explore different pre- and postprocessing strategies, and related label encoding schemes, and discuss their suitability for real-time transcription. Evaluating the adaptions on the MAESTRO dataset, we find a drop in transcription accuracy due to strictly causal processing as well as a tradeoff between the preprocessing latency and prediction accuracy. We release our system as a baseline to support researchers in designing models towards minimum latency real-time transcription.

1. INTRODUCTION

Automatic music transcription (AMT) is the task of transforming audio signals into their symbolic music representation, and is commonly referred to as one of the holy grails in Music Information Retrieval (MIR), given its role in linking the audio and symbolic domain, as well as its relevance to various downstream tasks and musical applications [1, 2]. The transcription of piano solo music is among the most extensively studied tasks, driven by the instrument's well-defined onset characteristics and the availability of large-scale, strongly labeled training data [1]. Consequently, research on automatic piano transcription has seen substantial progress. Notable contributions include [1–4], with the former two setting new benchmarks by leveraging larger architectures, increased model complexity, extended training, and a novel regression-based target encoding approach, leading to higher-resolution piano transcription.

Progress in automatic piano transcription has focused almost exclusively on offline methods, with only a few investigations attempting to solve this task online [5–7]. These online systems typically reconfigure offline approaches for block-wise updates while keeping audio representations and network structure, and achieve latencies between 128 and 320 ms. Latency values in the hundreds of milliseconds are suitable for musical applications such as subtitling, page turning, or visualizations. However, most interactive musical applications require lower latencies, e.g., for digital instruments, a commonly named bound is 10 ms [8–10], and for networked ensemble playing latencies of up to 30 ms are acceptable [11–13]. A real-time transcription model should enable fluent musical interaction, for which we take 30 ms as a minimal requirement, and 10 ms as a goal for imperceptible latency. Latency stems from various sources: audio buffering, preprocessing, model inference and postprocessing. For an automatic piano transcription system to perform inference in real-time, it must minimize latency in all sources. It should adhere to strict causality requirements in both the model architecture and the postprocessing, ensuring that both rely exclusively on past information.

In this work, we adapt a state-of-the-art model for real-time transcription towards minimal latency. We do so by allowing only causal processing within the model and reducing computational load by sharing computations across core components of the model. We further investigate the latencies incurred by widely used pre- and postprocessing strategies, and explore options to mitigate these using a combination of adapted STFT processing, label encoding schemes, loss functions, and causal post-processing. Our contribution is two-fold: First, we examine the sources of latency and causality violations in the current state-of-the-art system in online piano transcription, and propose and evaluate changes in the modeling and pre- and postprocessing stages for efficient real-time transcription. Second, we provide an open-source basis for a low-latency, real-time automatic piano transcription, inviting further research and development in this area.

The remainder of this article is structured as follows: In Section 2 we point to related work, one of which will form the our starting point for our adaptations towards minimum latency, which we will therefore focus on in greater



© P. Hu, S. Peter, J. Schlüter and G. Widmer. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** P. Hu, S. Peter, J. Schlüter and G. Widmer, "Exploring System Adaptations for Minimum Latency Real-Time Piano Transcription", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.,* Daejeon, South Korea, 2025.

detail in Section 3. In Section 4 we present our latency-minimizing adaptations and report the experiments conducted to assess their effect transcription accuracy. We discuss the challenges and lessons learned, and an outlook for future work in Section 5.

2. RELATED WORK

As outlined in the previous section, both online and real-time transcription remain largely unexplored. We will discuss three noteworthy contributions [5–7].

Fernandez [6] proposes a purely convolutional modeling approach that focuses solely on onset and velocity prediction, achieving a latency between 4 and 9 seconds.

Kwon et al. [7, 14, 15] propose an autoregressive neural network for efficient online piano transcription. The architecture comprises two main components: an acoustic module consisting of a stack of convolutional layers with frequency-conditioned FiLM layers, and a note sequence module consisting of pitchwise LSTMs, a multi-state softmax output (for different note states: onset, sustain, re-onset, offset, and off), and an autoregressive connection from the note state output of the previous time step to the current sequence module input. The authors propose various architectures that balance accuracy and latency. Overall, their models achieve latencies from 128 to 320 ms.

Kusaka and Maezawa [5] introduce Mobile-AMT, a framework designed to tackle both real-time processing and generalization to unseen recording environments in automatic piano transcription. They optimize a state-of-the-art offline transcription model [2] by replacing its conventional convolutional components with lightweight, efficient alternatives [16] and train it using a data augmentation scheme that simulates four distinct acoustic distribution shifts. The resulting model sets the current state of the art in online automatic piano transcription, achieving F1 scores comparable to offline state-of-the-art models while being robust to in-the-wild recordings. Its latency is reported as 174 ms – but an apparent oversight in the architecture increases latency to 10 s. We use this method as our starting point and detail it in the next section.

3. STARTING POINT

As Mobile-AMT [5] represents the current state of the art in real-time piano transcription, we use this method as the foundation and reference method for our adaptations towards minimum-latency transcription. To provide context for these modifications, we first outline the structure of the model, and particularly focus on the modeling aspects that violate causality requirements for real-time systems and necessitate adaptation.

3.1 Model Architecture

Mobile-AMT [5] is a lightweight adaptation of the state-of-the-art offline piano transcription model by Kong et al. [2]. It replaces all conventional convolutional blocks with MobileNet [16] equivalents, which consist of depthwise separable convolutions that reduce computational complexity while maintaining representational power. Additionally, Mobile-AMT removes all bi-directional flows in recurrent model layers, and drops one of originally four acoustic model core stacks (the one for note offset prediction, which is instead conditioned on the frame and onset output [1]). All optimization and activation layers are retained from the original offline model. With these modifications, the authors report a resulting latency of 174 ms, and argue their model to be capable of real-time inference.

Apart from depthwise separable convolutions, Mobile-AMT also adopts MobileNet's Squeeze-and-Excitation (SE) layers to dynamically recalibrate channel-wise features. The squeeze operation involves global pooling over all spatial dimensions, and therefore relies on information from the entire feature map, making the operation non-causal. As Mobile-AMT processes its input in 10-second blocks, the squeeze operation adds 10 seconds of latency, which is not accounted for in the authors' calculations.

3.2 Postprocessing

Mobile-AMT uses the same regression target encoding format as its underlying offline transcription model [2], resulting in incrementally increasing and decreasing onset and offset targets over a sequence of frames instead of binary, pointwise classification targets.

While this target encoding format allows for high, sub-frame resolution for onset and offset detection, it necessitates non-causal postprocessing, as the detection of an onset/offset relies on both past and future frames. The authors of Mobile-AMT adopt the same postprocessing strategy as [2], and account for it in their overall latency calculation.

3.3 Training and Evaluation Setup

Mobile-AMT is trained on 10-second segments of 16 kHz audio, which are transformed into 229-binned mel spectrograms after an STFT with a 2048-sample Hann window and a hop size of 320 samples. The loss function for the onset, offset and frame targets is binary cross-entropy (BCE), and velocities are trained using mean squared error (MSE).

Mobile-AMT proposes a data augmentation scheme for training to enhance robustness to real-world, in-the-wild recordings, therefore training duration depends on the data augmentations applied. For the non-augmented baseline, Mobile-AMT is trained for 3000 epochs on the MAESTRO dataset [1] with a batch size of 16 and uses the Adam optimizer with a learning rate of 0.001 annealed to 0 following a cosine schedule [12].

4. REAL-TIME ADAPTATIONS

As our main contribution, we implement and evaluate adaptations of the starting point that aim to reduce the system's latency. We form three groups of experiments:

[1] The offset prediction is omitted during inference, but no additional details are provided on whether or how the postprocessing is adjusted to account for this missing information.

adapting the training and postprocessing, adapting the audio preprocessing, and adapting the network architecture. We conduct each experiment with a reduced training time of 500 epochs (30k updates, approximately one-sixth of the total training time reported in our reference method [5]) to save computational resources, as most effects become evident already during the early training stages. We perform one final comparison of a model in which we combine selected modifications, and compare this to our reference method, with both models trained for 2000 epochs.

For all our experiments, unless otherwise noted, we train on 3-second segments of audio at 16 kHz, transformed into 229-bin mel spectrograms after an STFT with a Hann window of 2048 and a hop size of 160 samples. We double the frame rate compared to Mobile-AMT to allow for more frequent updates, and therefore reduced latency.

Apart from the training duration and data encoding, we follow the same training scheme as used in the non-augmented setup in our reference method, i.e., all adapted models are trained and evaluated on the MAESTRO v3.0 dataset using *mir_eval* [17]. As our work focuses on minimizing delay in real-time transcription, we mostly focus on the note onset metrics. Except for the final comparison, we only evaluate on the validation set. Since our goal is to achieve minimal latency for real-time musical interactions, we assess inference performance on stricter timing tolerances (10–30 ms) to reduce the mismatch between system evaluation and target performance.

4.1 Training and Postprocessing

In Mobile-AMT, the training target for an onset or offset extends over multiple time steps, forming a triangle centered on the target annotation. Originally proposed by Kong et al. [2], such triangles allow expressing positions at a higher resolution than the frame rate. However, predicting such triangles requires lookahead in the model, as the output must increase before the actual event. Furthermore, interpreting the predictions requires lookahead in postprocessing in order to find each triangle's maximum. As a preparation for switching to a causal version of Mobile-AMT with causal postprocessing, we thus replace the triangular targets with binary targets that are active only at the frame that is closest to an annotation.

As the changed label encoding results in a heavily imbalanced binary classification task, particularly for onset and offset targets, we try weighting their positive occurrences with a factor of 10 in the binary cross-entropy loss [18]. Additionally, pointwise classification targets penalize small temporal deviations more strongly than triangular targets. To account for this, we test applying a shift-tolerant loss recently proposed for beat detection [19], with a tolerance of ± 1 frame ($\pm$ 10 ms at our frame rate).

We also modify the postprocessing to ensure that the prediction for the current frame relies only on past information. First, we binarize the output probabilities: for onset and offset targets, an activation is detected when the current frame exceeds a given threshold while the previous frame does not. A frame activation is recorded if the current frame surpasses the threshold. Next, we eliminate re-onsets of the same pitch that occur within a predefined minimum re-onset distance. Finally, we determine the offset of an active note based on the earlier occurring one of either frame inactivity or offset activity.

Tolerance	10 ms	20 ms	30 ms
`Exp.`	Postprocessing onset threshold: 0.45		
`TP1`	9.58 ±2.29	25.80 ± 4.83	47.44 ± 7.30
`TP2`	29.84 ±8.37	47.64 ± 11.76	50.28 ± 11.83
`TP3`	13.95 ±4.91	34.34 ± 8.05	45.10 ± 8.33
`TP4`	27.24 ±7.47	55.49 ± 9.73	64.95 ± 10.06
`TP5`	16.00 ±5.32	41.02 ± 8.50	55.40 ± 8.53
`Exp.`	Postprocessing onset threshold: 0.55		
`TP1`	13.42 ±2.60	35.07 ± 5.31	56.02 ± 8.14
`TP2`	23.56 ±8.41	35.50 ± 11.44	36.87 ± 11.49
`TP3`	17.04 ±5.52	40.54 ± 8.26	51.46 ± 8.29
`TP4`	27.42 ±7.52	54.95 ± 10.18	63.67 ± 10.72
`TP5`	17.46 ±5.59	44.19 ± 8.53	58.68 ± 8.61
`Exp.`	Postprocessing onset threshold: 0.65		
`TP1`	19.10 ±3.12	43.88 ± 7.02	59.93 ± 9.77
`TP2`	14.32 ±6.70	20.52 ± 8.68	21.07 ± 8.70
`TP3`	20.70 ±5.90	46.48 ± 8.28	56.72 ± 8.28
`TP4`	27.18 ±7.65	53.65 ± 10.71	61.55 ± 11.36
`TP5`	18.74 ±5.80	46.80 ± 8.60	61.05 ± 8.89

Table 1. Comparison of note onset F1 scores (mean ± std each over three experimental runs) on the MAESTRO v.3 validation set across three onset tolerances for different target encodings, (weighted) loss functions and onset thresholds in postprocessing.

We test the change in label encoding in combination with different (weighted) loss functions in five different experimental setups: In `TP1` we train our reference model with the original loss functions and regression target encoding scheme as proposed by the authors [5]. In `TP2` we use classification targets, and in `TP3` we additionally apply a weight factor 10 on both the onset and offset targets. Finally, we apply the shift-tolerant BCE loss (with a tolerance of $\pm$10 ms), either unweighted (`TP4`) or weighted again by a factor of 10 (`TP5`). All five setups use our strictly causal postprocessing described above. The model architecture remains as proposed in Mobile-AMT.

Table 1 lists the note onset F1 scores on the MAESTRO v3.0 validation set for the different training targets and loss functions, combined with different onset threshold values applied during postprocessing, as evaluated on different onset tolerance thresholds. We choose to evaluate at lower tolerance thresholds as they reflect a real-time system's practical responsiveness better than the commonly used $\pm$ 50ms do, which might mask latency issues by crediting the model for detections that would feel delayed to a user in an interactive scenario. Learning pointwise binary targets without any further modification (`TP2`) proves superior to short sequential regression (`TP1`), provided a low enough onset threshold during postprocessing. The effect of lower onset detection thresholds is somewhat mitigated by weighting positive labeled examples higher (`TP3`), and virtually fully eliminated by using a shift-tolerant loss (`TP4`). There is no further improvement when

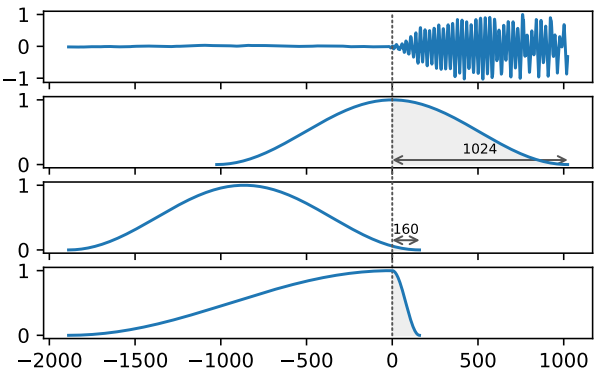


Figure 1. Mobile-AMT centers STFT windows on the time point to predict for, incurring a delay of 1024 samples. We shift the window to reduce the delay to 160 samples, and change the window function to better use that limited amount of future information.

combining weights and shift-tolerance (`T5`).

For the next experiments, we use binary targets with weights. While the shift-tolerant loss performed favorably here, our goal is to use a causal model, for which shift tolerance could result in systematically delayed predictions.

4.2 Audio Preprocessing

Mobile-AMT processes audio in spectrogram frames of 2048-sample windows centered on the time points to predict events for. Thus, even with a causal model that does not process information from future frames, in real-time inference, every time a complete audio buffer of 2048 samples is filled, inference will be triggered to predict events that are already 1024 samples (64 ms) in the past. Figure 1 illustrates this: the top row shows an audio waveform, the second row a typical STFT window centered at the onset. Shorter filters reduce this delay, as the delay is fixed to half the filter length, but this comes at the cost of lower frequency resolution, which we should avoid: With a 2048-sample STFT at 16 kHz, the bin width is 7.8 Hz, which is already too coarse to achieve semitone precision for the lowest piano notes (A0 at 27.5 Hz, B♭0 at 29.14 Hz).

We can however reduce this delay to a lower number n_s of samples (e.g., 160 samples or 10 ms) while keeping the window length and frequency resolution unchanged, by shifting windows so they end n_s samples after their reference point instead of being centered. The third row in Figure 1 shows the Hann window shifted to $n_s = 160$ samples. The Hann window strongly attenuates the boundaries, the right one of which now contains highly relevant information for the prediction. To mitigate this unwanted attenuation, we can replace the Hann window with an asymmetric window that tapers $(2048 - n_s)$ samples before and n_s samples after the reference point. The last row in Figure 1 illustrates this windowing function for a 1888/160 sample asymmetry. Note how we keep more information from the incoming samples in the gray shaded area under the window function, albeit at the cost of increasing spectral leakage (by about 20 dB for $n_s = 160$).

Tolerance	10 ms	20 ms	30 ms
`H1` Hann 64 ms	17.43 ± 5.10	34.65 ± 7.40	39.86 ± 7.45
`H2` Hann 10 ms	0.00 ± 0.01	0.00 ± 0.02	0.04 ± 0.10
`T1` asym. 10 ms	22.25 ± 5.17	25.21 ± 5.20	25.84 ± 5.18
`T2` asym. 20 ms	28.61 ± 7.09	33.76 ± 7.01	34.65 ± 6.89
`T3` asym. 30 ms	27.61 ± 6.79	37.91 ± 7.54	39.43 ± 7.33
`T4` asym. 40 ms	24.39 ± 6.50	37.51 ± 7.81	39.87 ± 7.60
`T5` asym. 50 ms	20.99 ± 6.02	36.88 ± 7.75	40.47 ± 7.59
`ST` asym. 10 ms	0.41 ± 0.40	1.57 ± 0.82	11.86 ± 4.25

Table 2. Note onset F1 scores on the MAESTRO v.3 validation set for different windowing functions. The last row additionally uses a shift-tolerant training loss.

To experiment with different windowing configurations for reducing the delay in audio preprocessing, we modify our reference method to apply only causal processing, as allowing the model access to future frames would render our interventions meaningless. Specifically, we make each convolution causal, so the model's receptive field of 9 frames extends 8 frames into the past, rather than splitting 4 frames into the past and 4 frames into the future. Additionally, we remove the Squeeze-Excitation layers of the MobileNet V3 blocks, which perform global average pooling over both past and future frames in an excerpt.

Table 2 shows our results. The original centered Hann window with our causal model (`H1`) performs worse than our non-causal starting point (`TP3` in Table 1). Shifting the Hann window from a delay of 64 ms to a delay of 10 ms (`H2`) seems to completely attenuate usable information in the frames. Using an asymmetric window (`T1`) improves performance, but still falls behind the centered Hann window. Successively increasing the delay up to 50 ms, we see a strong improvement (`T2`–`T5` and Figure 2). For a delay of 30 ms or more, we match performance of the centered Hann window at an evaluation onset tolerance of 30 ms. For stricter tolerances, shifted asymmetric windows of 20 ms delay or more surpass the centered Hann window.

We also take the chance to investigate how a shift-tolerant loss of ±1 frame affects results for the causal model. The loss could allow the model to systematically predict events one frame (10 ms) later than annotated. Surprisingly, using an asymmetric window with 10 ms of delay, we find that the shift-tolerant loss (`ST`) performs on par with 30 ms delay (`T3`) when admitting an evaluation tolerance of 50 ms (not shown in table), but breaks down with any stricter tolerance (as seen in the table).

For the third group of experiments, we keep the strictest setting with asymmetric windows at a delay of 10 ms.

4.3 Model Architecture

In our final group of experiments, we investigate architectural modifications to our reference model. The architecture of Mobile-AMT consists of three acoustic stacks, each consisting of recurrent convolutional blocks. Each stack learns a (onset, frame or velocity) target. For some targets, the outputs of multiple stacks are concatenated to condition the final predictions. Compared to their reference offline model [2], Mobile-AMT omits the acoustic stack for the offset target, reusing the stack of the frame target.

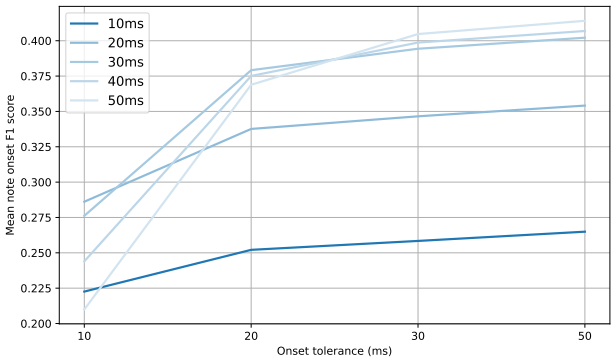


Figure 2. Note onset F1 scores (means only) for different window delays and onset tolerance thresholds.

Tolerance	10 ms	20 ms	30 ms
	Note onset F1 mean ± std		
A1	20.23 ± 4.29	23.03 ± 4.25	23.75 ± 4.24
A2	25.77 ± 4.65	30.66 ± 4.98	31.79 ± 4.93
A3	21.59 ± 4.79	24.48 ± 4.83	25.14 ± 4.80
A4	21.53 ± 4.31	24.51 ± 4.24	25.22 ± 4.19
A5	19.44 ± 4.20	22.39 ± 4.34	23.12 ± 4.30
A6	25.82 ± 4.48	30.52 ± 4.51	31.56 ± 4.44
	Note onset and offset F1 mean ± std		
A1	3.25 ± 1.31	5.13 ± 2.54	7.20 ± 3.51
A2	3.56 ± 1.29	6.17 ± 2.80	8.39 ± 4.01
A3	5.84 ± 1.41	5.84 ± 2.48	7.71 ± 3.43
A4	5.94 ± 1.31	5.94 ± 2.48	7.89 ± 3.49
A5	4.84 ± 1.11	4.84 ± 2.36	6.61 ± 3.32
A6	6.79 ± 1.15	6.79 ± 2.53	8.89 ± 3.59

Table 3. Note onset and onset-and-offset F1 scores on the MAESTRO v.3 validation set across three onset tolerances for architectural modifications and input representations.

Our experiments involve the following adaptations, each tested independently: First, in A1 we (re)introduce a separate offset acoustic stack to explore whether and how it improves offset label prediction. In A2 we remove the velocity conditioning on the onsets. Next, we examine whether further streamlining the architecture by sharing a fourth (A3), half (A4) or all (A5) of the convolutional blocks in the model's acoustic stacks affects performance. Lastly, in A6 we examine the effect of training on the original 10 seconds sequence length.

Table 3 presents the note onset and note onset-and-offset F1 scores of the model and data adaptations on the MAESTRO validation set. Overall, it is evident that the combined impact of binary, heavily imbalanced pointwise targets, causal modeling, and shifted asymmetric window results in a significantly harder learning problem, with the same training duration (500 epochs) leading to significantly poorer scores than the base case (TP1 in Table 1). However, across all experimental setups in Section 4.1 compared to the current one, all our causal modifications demonstrate significantly stronger robustness to decreasing tolerance thresholds, which is important to guarantee low latency in predictions, and therefore appear promising for further training.

Furthermore, when comparing all model architecture modifications (A1–5) on note onset and note-onset-and-offset F1 score, we observe two unexpected model behaviours: First, adding a separate offset stack (A1) does not improve offset prediction. As our postprocessing detects a note offset as the earlier of either offset activation or frame inactivation, we hypothesize that frame activity is sufficiently learned to compensate for the absence of an offset acoustic stack. Second, removing the velocity conditioning on onset prediction (A2) results in a strong improvement in onset prediction. Furthermore, sharing the acoustic stack across increasing proportions (A3–5) does not appear to hinder the model's ability to learn meaningful representations. Finally, experiment A6 suggests that the model benefits from the larger contextual window.

4.4 Final comparison

For our final comparison, we proceed with the following data and model configurations: we continue with the (160 samples) shifted asymmetric window for the STFT (T1 in Sec. 4.2), remove the velocity conditioning (A2) and share all convolutional layers in the acoustic stack across all targets (A5). Mobile-AMT uses the original non-causal postprocessing described in Section 3.2, while our model use the causal postprocessing introduced in Section 4.1.

Table 4 summarizes the results over different onset (and offset) thresholds for note onset and onset-and-offset metrics. As expected, Mobile-AMT outperforms our modified causal model across all metrics, with a significant margin. Upon reviewing all experiments conducted, we conclude that the largest performance drops are attributed to the shifted window function and the causal convolutions in our model. When comparing Mobile-AMT and our adapted model across various onset tolerance thresholds, we observe, similar to the previous experiment, that while our modified causal model predicts fewer targets with lower accuracy overall, it demonstrates higher precision and robustness when evaluated at stricter timing tolerances.

5. DISCUSSION AND OUTLOOK

In this work, we investigate whether and how the current state of the art in real-time piano transcription can be adapted to achieve minimum-latency automatic piano transcription suitable for real-time musical interaction.

What latency is suitable cannot be answered universally, so our choice of 10–30 ms is worthy of discussion. While 10 ms is suggested in digital instrument design [8–10], thresholds for latency perception vary depending on the musical situation, task, and instrument: for percussive digital instruments, decreased ratings for values of 20 ms and above were found [20], instrument-specific thresholds between below 10 ms and 40 ms are reported in a live monitoring setting [21], and about 30 ms were found for gestural control [22]. Offsets as low as 6 ms may be perceived in simple isochronously spaced stimuli [23], while other researchers found just noticeable latency differences at 27 ms and higher [24]. Transcription-enabled

		Note Onset			Note Onset with Offset		
Model	**Tol. (ms)**	**Precision**	**Recall**	**F1**	**Precision**	**Recall**	**F1**
Causal-AMT	10	43.51 ± 6.87	25.60 ± 8.64	31.55 ± 7.76	6.86 ± 2.15	4.11 ± 2.03	5.03 ± 2.08
Mobile-AMT	10	22.70 ± 5.82	15.57 ± 2.96	18.26 ± 3.51	3.27 ± 1.19	2.32 ± 0.94	2.69 ± 1.00
Causal-AMT	20	50.86 ± 7.52	29.71 ± 9.11	36.70 ± 7.96	10.89 ± 3.91	6.34 ± 2.90	7.86 ± 3.16
Mobile-AMT	20	59.28 ± 7.41	41.87 ± 8.44	48.52 ± 6.96	11.03 ± 4.15	7.98 ± 3.73	9.16 ± 3.84
Causal-AMT	30	51.85 ± 7.49	30.24 ± 9.07	37.38 ± 7.85	14.49 ± 5.72	8.33 ± 3.93	10.37 ± 4.42
Mobile-AMT	30	81.17 ± 6.29	57.84 ± 12.79	66.80 ± 9.78	18.42 ± 6.89	13.27 ± 6.09	15.26 ± 6.30

Table 4. Final comparison between our implementation of Mobile-AMT and our modified minimal-latency, strictly causally adapated model.

real-time applications like interactive accompaniment or generative improvisation are more akin to ensemble playing than direct instrument control. In networked musical contexts, researchers typically aim for 20–30 ms of latency to meet performance conditions that mirror traditional in-person ensembles [11, 12]. However, studies also found that musicians may be able to compensate for latencies up to 50 ms [13,25] or even 100 ms [26] for one piano piece, a value that was deemed "neither musical nor interactive" in another study [27]. A real-time transcription model should not only be tolerable but enable fluent musical interaction, so we took 30 ms as a minimal requirement, and 10 ms as a goal for imperceptible latency.

We investigate multiple adaptations to reduce latency, including label encoding with causal postprocessing, shifted asymmetric window functions during preprocessing, and architectural modifications that enforce causal processing within the model. Additionally, we reduce the model size (from 320 to 160 GFLOPs for 3 seconds of input) by sharing computations across core model components for all targets.

In a first set of experiments, we assess the impact of regression versus classification loss encodings for non-causal models. The original regression targets only make sense in conjunction with a lookahead as the targets begin to increase several frames before the actual onset which is impossible for a causal model to predict. To mitigate the cost in training stability and accuracy incurred by localized, causal-ready targets, we experiment with loss functions that weight the active frames over the inactive frame to combat label imbalance, and loss functions that are tolerant to small temporal shifts. We find that the weighted classification losses approximate the baseline, and the shift-tolerant losses reach the same level in the absence of targets requiring lookahead.

In a second experiment, we investigate the delay incurred by the computation of audio feature representations. Specifically, we look at STFT windows and their corresponding centered targets. Transcription requires high frequency resolution for pitch estimation which requires large windows. Centering the targets results in an often overlooked delay of half the window length, 64 ms in our case. We test configurations of shifted windows along with asymmetric windowing functions that do not attenuate the most recent samples. We find that aggressively shifted windows at 10 ms do deteriorate the transcription accuracy by a lot, yet at 30 ms, we reach comparable performance to an unshifted causal model. Here, a shift-tolerant loss does not improve performance. At the same time, configuring the model architecture for strictly causal processing also deteriorates performance with respect to the baseline with more than 100 ms of lookahead.

In a third experiment, we assess different model architectures and their impact. We observe that sharing the convolutional components of the acoustic stack across different target types proves beneficial. We hypothesize that the local acoustic features captured in the convolutional layers of the acoustic model can be effectively learned independently of sequential information, making them invariant to the target type. Furthermore, removing the velocity conditioning on the onsets strongly improves the accuracy of onset predictions.

Overall, we find that we can compensate well for algorithmic issues: we can scale the model and use lookahead-free targets without a major drop in performance. What proves difficult, however, is to render the model strictly causal and to effectively process the incoming audio without loss of relevant information. For a latency of 10 ms, it would be required that the model predicts pitches with at most 10 ms of incoming audio samples. For onsets of the lowest two octaves on the piano, this means that there is not even a full period of the fundamental frequency present in the samples, and predictions may need to rely on harmonic partials. Along with the transient phase and the consequently blurry STFT frame, this leads to an increasingly hard transcription task. We hope that these findings and pinpointed challenges will contribute to future research on real-time, minimum latency automatic piano transcription.

While this study primarily focuses on the algorithmic performance and robustness of a real-time transcription model, we acknowledge that a detailed analysis of processing time—including both network inference and preprocessing—across different hardware platforms remains an important area for future work to allow for the practical deployment of a real-time transcription system in real-world scenarios. Likewise, we want to take a closer inspection into the design of the underlying window function and filter bank, in order to find an appropriate balance between reducing prediction delay and increasing future context, all while maintaining the desired STFT properties. Lastly, across all experimental groups, our system adaptations consistently outperformed the baseline at lower timing tolerance, which we consider a desirable property worthy of further investigation.

6. ACKNOWLEDGEMENTS

This research acknowledges support by the European Research Council (ERC), under the European Union's Horizon 2020 research and innovation programme, grant agreement No. 101019375 *Whither Music?*. The LIT AI Lab is supported by the Federal State of Upper Austria.

7. REFERENCES

[1] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, "Enabling factorized piano music modeling and generation with the MAESTRO dataset," in *International Conference on Learning Representations*, 2019.

[2] Q. Kong, B. Li, X. Song, Y. Wan, and Y. Wang, "High-Resolution Piano Transcription with Pedals by Regressing Onset and Offset Times," *IEEE/ACM Transactions on Audio Speech and Language Processing*, vol. 29, pp. 3707–3717, 2021.

[3] S. Sigtia, E. Benetos, and S. Dixon, "An end-to-end neural network for polyphonic piano music transcription," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 24, no. 5, pp. 927–939, 2016.

[4] R. Kelz, S. Böck, and G. Widmer, "Deep polyphonic adsr piano note transcription," in *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019, pp. 246–250.

[5] Y. Kusaka and A. Maezawa, "Mobile-AMT: Real-Time Polyphonic Piano Transcription for In-the-Wild Recordings," in *2024 32nd European Signal Processing Conference (EUSIPCO)*. IEEE, 2024, pp. 36–40.

[6] A. Fernandez, "Onsets and Velocities: Affordable Real-Time Piano Transcription Using Convolutional Neural Networks," in *2023 31st European Signal Processing Conference (EUSIPCO)*. IEEE, 2023, pp. 151–155.

[7] T. Kwon, D. Jeong, and J. Nam, "Towards Efficient and Real-Time Piano Transcription Using Neural Autoregressive Models," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2024.

[8] D. Wessel and M. Wright, "Problems and prospects for intimate musical control of computers," *Computer music journal*, vol. 26, no. 3, pp. 11–22, 2002.

[9] A. P. McPherson, R. H. Jack, and G. Moro, "Action-sound latency: Are our tools fast enough?" in *16th International Conference on New Interfaces for Musical Expression, NIME 2016, Griffith University, Brisbane, Australia, July 11-15, 2016*. nime.org, 2016, pp. 20–25. [Online]. Available: https://doi.org/10.5281/zenodo.3964611

[10] F. Caspe, J. Shier, M. Sandler, C. Saitis, and A. McPherson, "Designing neural synthesizers for low latency interaction," *arXiv preprint arXiv:2503.11562*, 2025.

[11] L. Turchet and C. Rottondi, "On the relation between the fields of networked music performances, ubiquitous music, and internet of musical things," *Personal and Ubiquitous Computing*, vol. 27, no. 5, pp. 1783–1792, 2023.

[12] E. Lakiotakis, C. Liaskos, and X. Dimitropoulos, "Improving networked music performance systems using application-network collaboration," *Concurrency and Computation: Practice and Experience*, vol. 31, no. 24, p. e4730, 2019.

[13] E. Chew, R. Zimmermann, A. A. Sawchuk, C. Kyriakakis, C. Papadopoulos, A. François, G. Kim, A. Rizzo, and A. Volk, "Musical interaction at a distance: Distributed immersive performance," in *Proceedings of the MusicNetwork Fourth Open Workshop on Integration of Music in Multimedia Applications*. MusicNetwork Barcelona, 2004, pp. 15–16.

[14] T. Kwon, D. Jeong, and J. Nam, "Polyphonic Piano Transcription Using Autoregressive Multi-State Note Model," in *The 21th International Society for Music Information Retrieval Conference (ISMIR)*. International Society for Music Information Retrieval, 2020.

[15] D. Jeong and S. Telecom, "Real-time automatic piano music transcription system," in *Late Breaking Demo. International Society for Music Information Retrieval*, 2020, pp. 4–6.

[16] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan *et al.*, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 1314–1324.

[17] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, D. P. Ellis, and C. C. Raffel, "MIR_EVAL: A Transparent Implementation of Common MIR Metrics." in *ISMIR*, vol. 10, 2014, p. 2014.

[18] R. M. Bittner, J. J. Bosch, D. Rubinstein, G. Meseguer-Brocal, and S. Ewert, "A lightweight instrument-agnostic model for polyphonic note transcription and multipitch estimation," in *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, Singapore, 2022.

[19] F. Foscarin, J. Schlüter, and G. Widmer, "Beat this! Accurate beat tracking without DBN postprocessing," *arXiv preprint arXiv:2407.21658*, 2024.

[20] R. H. Jack, A. Mehrabi, T. Stockman, and A. McPherson, "Action-sound latency and the perceived quality of digital musical instruments: Comparing professional

percussionists and amateur musicians," *Music Perception*, vol. 36, no. 1, pp. 109–128, 09 2018. [Online]. Available: https://doi.org/10.1525/mp.2018.36.1.109

[21] M. Lester and J. Boley, "The effects of latency on live sound monitoring," *Journal of the Audio Engineering Society*, no. 7198, october 2007.

[22] T. Mäki-Patola and P. Hämäläinen, "Latency tolerance for gesture controlled continuous sound instrument without tactile feedback," in *Proceedings of the 2004 International Computer Music Conference, ICMC 2004, Miami, Florida, USA, November 1-6, 2004*. Michigan Publishing, 2004. [Online]. Available: https://hdl.handle.net/2027/spo.bbp2372.2004.032

[23] A. Friberg and J. Sundberg, "Time discrimination in a monotonic, isochronous sequence," *The Journal of the Acoustical Society of America*, vol. 98, no. 5, pp. 2524–2531, 1995.

[24] A. Schmid, M. Ambros, J. Bogon, and R. Wimmer, "Measuring the just noticeable difference for audio latency," in *Proceedings of the 19th International Audio Mostly Conference: Explorations in Sonic Cultures, AM 2024, Milan, Italy, September 18-20, 2024*, L. A. Ludovico and D. A. Mauro, Eds. ACM, 2024, pp. 325–331. [Online]. Available: https://doi.org/10.1145/3678299.3678331

[25] S. Dahl and R. Bresin, "Is the player more influenced by the auditory than the tactile feedback from the instrument," in *Proceedings of the Digital Audio Effects Conference (DAFx)*, 2001, pp. 6–9.

[26] A. A. Sawchuk, E. Chew, R. Zimmermann, C. Papadopoulos, and C. Kyriakakis, "From remote media immersion to distributed immersive performance," in *Proceedings of the 2003 ACM SIGMM Workshop on Experiential Telepresence*, ser. ETP '03. New York, NY, USA: Association for Computing Machinery, 2003, p. 110–120. [Online]. Available: https://doi.org/10.1145/982484.982506

[27] C. Bartlette, D. Headlam, M. Bocko, and G. Velikic, "Effect of network latency on interactive musical performance," *Music Perception*, vol. 24, no. 1, pp. 49–62, 09 2006. [Online]. Available: https://doi.org/10.1525/mp.2006.24.1.49